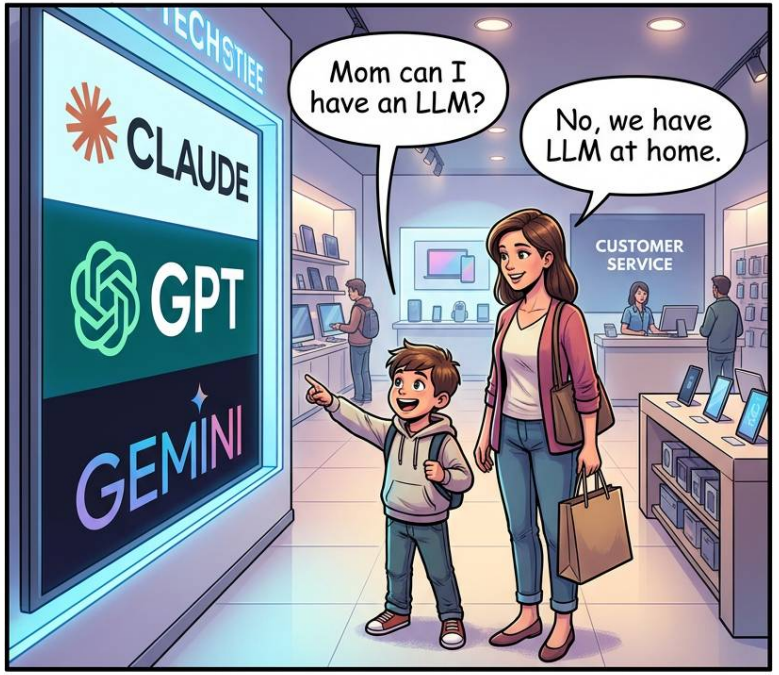
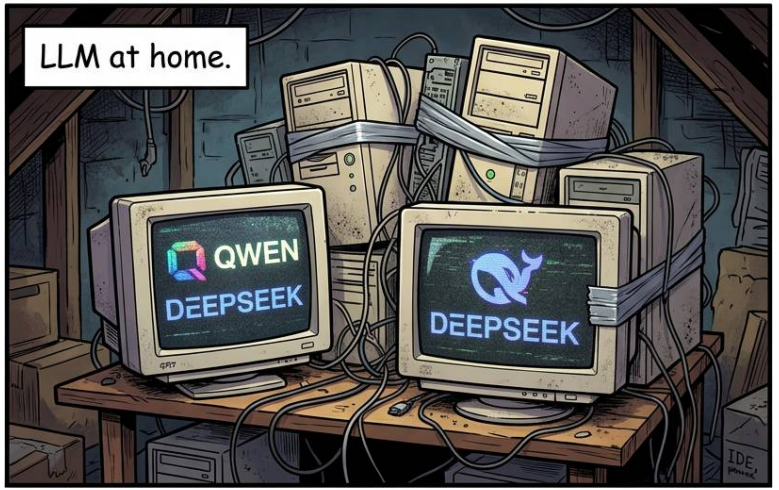
Agenda

- A bit of theory
  - Prefill
  - Decode
  - Optimization
  - Quantization
  - Parallel inference
- Inference engines
- Hardware
- Is it worth it?





Hosting LLM models

Jarek Jaryszew
claw-destine.com
Software developer for 20 years
ML since covid, gen AI since 2024
Also a bit of a hardware geek



Models

Currently most popular models for devices under 128GB:

- Qwen 3.8 Flash Next
- Qwen 3.8 27B
- Qwen 3.6 – few options for even smaller machines



Models which unfortunately grown a lot since the previous version:

- Deepseek V4.1 Flash
- GLM 5.3





Possibly interesting in future:

- Nvidia’s Nemotron
- Ling family

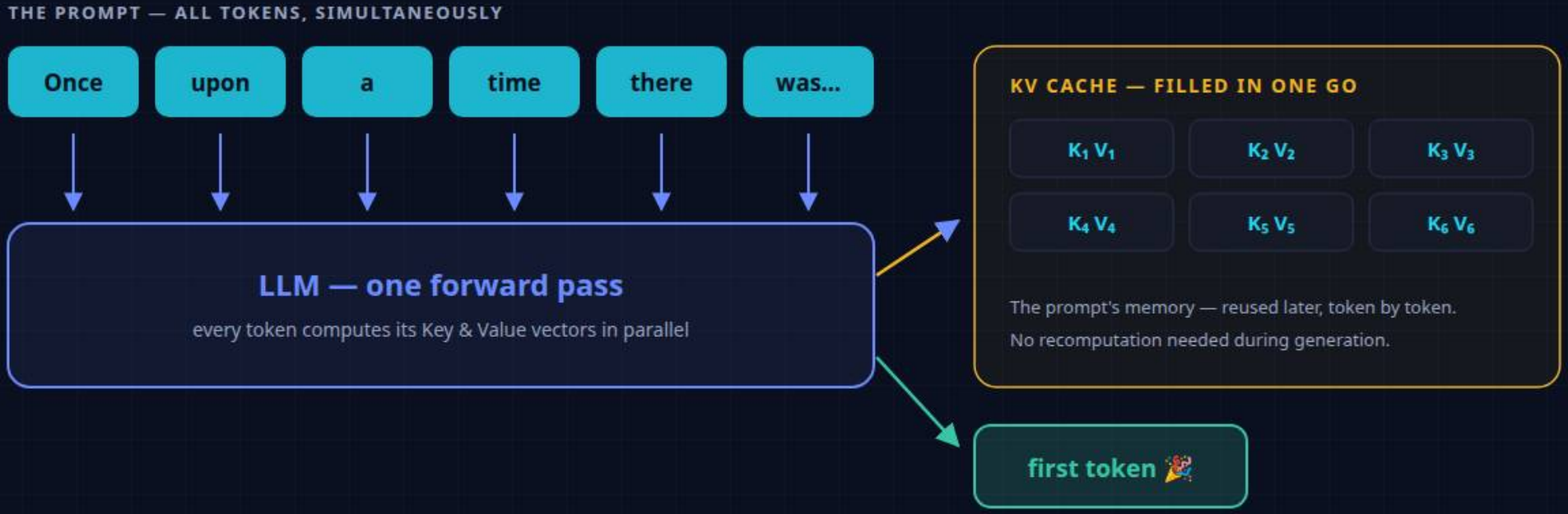




LLM INFERENCE · STEP 1

Prefill: read the whole prompt at once

All prompt tokens flow through the model in parallel — one big pass.



Character: parallel over all prompt tokens · compute-bound · determines time-to-first-token (TTFT).

Output: a full KV cache + the first generated token.

LLM INFERENCE · STEP 2

Decode: one token at a time

Each step feeds in the last token, reads the KV cache, emits one new token — and loops.



Character: sequential · memory-bound (reading the KV cache dominates) · sets tokens-per-second.

Cheap per step, but N steps = N passes — that's why generation feels slower than prompt reading.

Prefill optimisation – K/V cache

- Example
  - Imagine reasonable prefill 1000 TPS
  - Context lenght 256k
  - 256 seconds (over 4 minutes)!
- Solution: keep K/V cache after each inference
  - Skip prefill for previously decoded context
  - Order matters. If you change one message all the former has to be reprocessed

Decode optimization

- Model architecture
  - Baked into model before training
  - Most popular: MoE, n-grams
  - New: Causal Encoder-Decoder (Deepseek), Expert Swap (Strata)
- Speculative decoding
  - May be baked in or use a lightweight side-car model
  - That’s why you want to look for recipes
- Many methods may be combined

EXPLAINER · MIXTURE-OF-EXPERTS IN QWEN

What is Mixture-of-Experts (MoE)?

Many expert networks, but only a few handle each token — a router picks them.



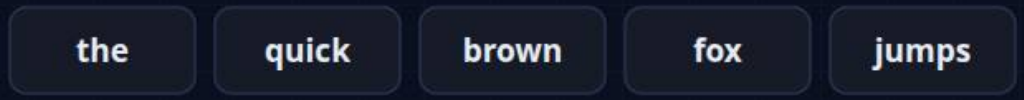
Why Qwen uses it: Qwen3.8-Flash-Next has 512 experts and routes each token to just 10 — a big model that computes like a small one.

EXPLAINER · N-GRAM EMBEDDING IN QWEN

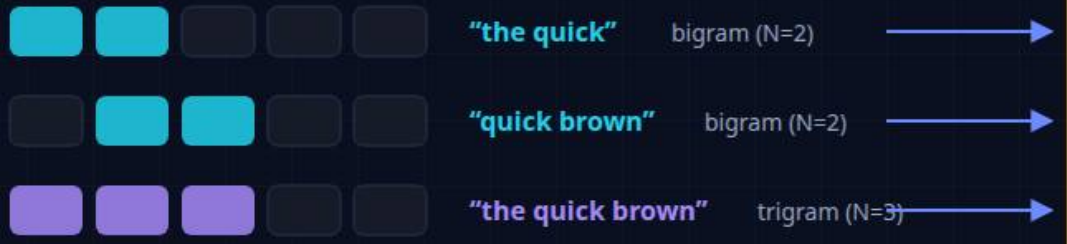
What is an N-gram?

A window of N adjacent words/tokens — a tiny fixed-size slice of local context.

A SENTENCE, TOKENIZED



SLIDE THE WINDOW, ONE TOKEN AT A TIME



LOOKUP TABLE · LEARNED VECTORS

"the quick"	[0.12, −0.87, … , 0.31]
"quick brown"	[−0.44, 0.09, … , 0.77]
"the quick brown"	[0.65, 0.21, … , −0.53]

Each n-gram key maps to one learned vector — a direct memory read.

Why Qwen uses it:

Qwen3.8-Flash-Next stores ~20M bigram + trigram entries (51B parameters) and looks them up at layer 2.

A lookup is a memory read, not a matrix multiply — huge capacity, almost no extra compute per token.

Speculative decoding

- MTP (Multi-Token Prediction): Uses auxiliary layers built directly into or paired with the main model to predict upcoming sequential tokens (t+1, t+2, etc.).
- DFlash (and DFlash 2): Uses a lightweight block-diffusion approach to draft an entire block of tokens in a parallel forward pass rather than one-by-one.
- EAGLE-3: advanced speculative decoding algorithm that speeds up large language model (LLM) text generation by using a lightweight draft model to predict multiple tokens ahead for single-pass verification by the target model.

Improvements keep coming

- Causal Encoder-Decoder – half of the model for prefill, half for decode
- Swapping experts during runtime between faster and slower GPUs or CPUs
- Loading layers/experts directly from NVE when needed (Tinycorp’s 16x RAID 0 or soon HBF – high bandwidth flash)

LLM QUANTIZATION · SLIDE 1

What gets quantized

A transformer has three big tensors — weights, activations, KV cache — and each trades something different.

WEIGHTS

W4A16 — the default move
decode is memory-bound



≈ 4× less memory traffic per token, activations still compute in FP16.

70B: 140 GB → ~38 GB fits one GPU

Near-free speedup at batch 1 · biggest payoff

ACTIVATIONS

W8A8 / W4A4 — harder
data, made at runtime



A few giant outlier values wreck a shared scale — everything else collapses to zero.

Throughput win at large batch

Needed to use INT8/FP8 tensor-core *math*, not just storage.

Fix: migrate outliers into weights, or rotate them away

KV CACHE

FP8 now routine,
2–4 bit = frontier



Dominates memory at long context: can exceed the weights themselves.

128K context: KV can be > weights

Per-token, per-channel scales · asymmetric K/V treatment

Rule of thumb: quantize weights first (free), KV cache when context is long, activations only when you need tensor-core math.

Naming you'll see: WxAy = weights x-bit, activations y-bit — W4A16, W8A8, W4A4.

LLM QUANTIZATION · SLIDE 2

Numeric formats: where the 16 levels sit

Same budget — 4 bits, 16 values per weight — but uniform, floating-point, or quantile spacing choose very different levels.

INT4 · uniform $w \approx \text{scale} \times q,\ q \in \{-8 \dots 7\}$

evenly spaced — resolution wasted near the extremes



FP4 (E2M1) · floating-point sign · 2-bit exponent · 1-bit mantissa

dense near zero, gaps grow with magnitude — matches weight + activation distributions



NF4 / i-quant tables · quantile-spaced levels = percentiles of a normal distribution

integers on disk, but each one indexes a float lookup table — best quality per bit at low bit-widths



Common to all of them — block scaling: 8 low-bit values → **1 float scale per block**

Block size is the accuracy / overhead knob: **INT ~128** · **MXFP4 32** · **NVFP4 16** — smaller block, closer fit, more metadata.

FP8 = E4M3 for weights · E5M2 when range matters · INT8 = older tensor-core path · FP16/BF16 = the "16" everything compresses from.

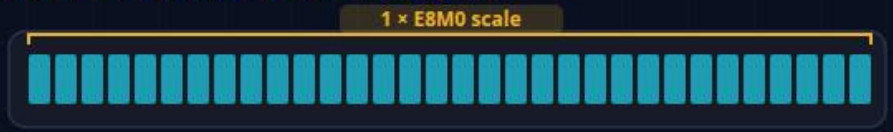
Weights are near-Gaussian and activations have outliers — that's why float formats and quantile tables beat uniform ints at 4 bits.

LLM QUANTIZATION · SLIDE 3

NVFP4 vs MXFP4: same 4 bits, different scales

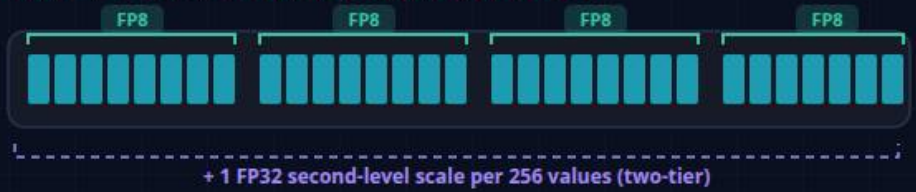
Both store E2M1 4-bit floats. The whole difference is how often they re-scale — and in what precision.

MXFP4 · OCP Microscaling standard



· 32 values share one scale
· scale is a **power of two only** (E8M0) → can land far from ideal → coarse
· ~4.25 bpw · open standard: AMD/Intel GPUs, native in **MI350 / Blackwell**

NVFP4 · NVIDIA format (Blackwell native)



· 16 values share a scale · scale is real FP8 E4M3 **(any value)** → finer
· ~4.5 bpw · **Blackwell tensor cores compute FP4 natively** — no dequant pass

Why finer, floating-point scales win



Size ladder on one GPU (70B)



bar = weight bytes only — KV cache & activations separate

Takeaway: FP4 was "too lossy" until hierarchical scaling fixed it — MX vs NV is the same idea at different scale density.

LLM QUANTIZATION · SLIDE 4

Decoding GGUF names

llama.cpp's labels are a grammar — family, bits, structure, size tier. Read it left to right.

ANATOMY OF "Q4_K_M"

Q4 — ~4-bit · real 4.86

_K — two-level blocks

_M — mixed: S < M < L

SAME GRAMMAR ELSEWHERE

IQ3_XXS i-quant · ladder XXS·XS·S·M·NL

Q4_0 / Q4_1 legacy · _0 symmetric, _1 + offset

_S → _M → _L more tensors bumped to higher bits

ACTUAL BITS / WEIGHT



What "K" buys you — one super-block of 256 weights:



8 sub-blocks × 32 values, each with its own finer-grained scale

The scales themselves get quantized and amortized over 256 values — accuracy of tiny blocks, overhead of big ones.

What "IQ" buys you:



stored int index → codebook of hand-picked, quantile-spaced float levels

Still integers on disk — but they're table indices, not grid points.

An **importance matrix** (imatrix) decides where the table spends resolution.

Integer storage, float compute: dequantize-to-FP16 runs inside the matmul kernel, on the fly — that's what makes these work everywhere.

In practice: Q4_K_M default · IQ4_XS when VRAM-tight · Q6_K / Q8_0 near-lossless · below Q3_K, quality falls off a cliff.

LLM QUANTIZATION · SLIDE 5

The algorithm names to drop

Every named method is an answer to one question: where does the error go when you round a weight?

PTQ post-training, minutes on a calibration set — everything below

QAT retrain with fake-quant in the loop — only needed < 3 bits or W4A4

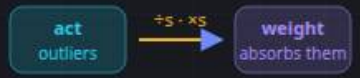
GPTQ — "compensate with the next weights"

round a column, measure its error, push it into
the remaining columns using a Hessian (2nd-order)
the original 4-bit GPU workhorse · needs calibration data

AWQ — "protect the ~1% that matter"

find salient channels by activation magnitude,
pre-scale weights so error avoids them. No backprop
usually > GPTQ at 4-bit · the default GGUF/serving rival

SmoothQuant — W8A8 on GPUs



equalizes the layer so both sides fit INT8 —
math unchanged, just rescaled by diag(s)

bitsandbytes · NF4 · QLoRA

int8 matmul + fp16 for outlier columns = LLM.int8()
NF4 = quantile-spaced levels (the GGUF IQ idea, earlier)
QLoRA: 4-bit frozen base + LoRA trains on one consumer GPU

QuaRot · SpinQuant — rotate the outliers away

multiply by a Hadamard rotation ($W \cdot H$, $H^T \cdot H = I$ so math
is identical) — outliers smear into a Gaussian shape
enables extreme W4A4KV4 — everything at 4 bits, even KV

QuIP# · AQLM · vector-quant

integers on a grid run out of steam ~3 bits →
learn codebooks: index into k-tuples of float vectors
2-bit territory · GGUF i-quants are the training-free cousins

How to say it in one breath: GPTQ/AWQ compress weights · SmoothQuant/QuaRot make activations survivable · codebooks push past 3 bits.

Serving today: AWQ or FP8 on vLLM · i-quants on a laptop · NVFP4 on Blackwell — same goal, different hardware eras.

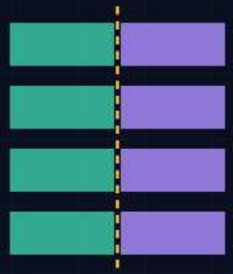
LLM INFERENCE PARALLELISM · SLIDE 1

Five ways to cut one model

Every parallelism scheme answers one question: which axis of the stack lands on which GPU?

TP · Tensor

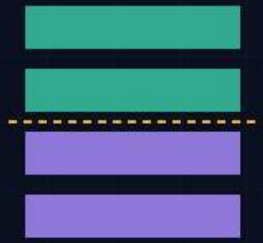
splits every weight matrix



all-reduce ×2 every layer

PP · Pipeline

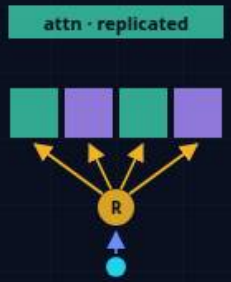
splits blocks of layers



1 hop at stage boundary

EP · Experts

whole experts, per GPU (MoE)



all-to-all per MoE layer

CP · Context

splits the token sequence



KV traffic across ranks

DP · Replicas

nothing — full copies



zero communication

GPU 0 · GPU 1 · cut / collectives · request / tokens

Rule of thumb: TP and PP shard a dense model; EP swaps matrix-sharding for expert-sharding when the FFN is sparse.

In vLLM flags: --tensor-parallel-size · --pipeline-parallel-size · --data-parallel-size · --enable-expert-parallel.

LLM INFERENCE PARALLELISM · SLIDE 2

What each cut costs

Decode phase, one token at a time: every scheme trades compute time for a different kind of communication.

TENSOR · TP

latency king,
bandwidth glutton



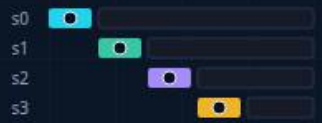
2 all-reduces per layer — ~80 layers
per generated token

Each GPU does 1/N of every matmul, then they must merge partial sums.

Gain: ~linear speedup to 8 GPUs on NVLink · bigger KV batch
Cost: comm latency-bound — net-negative past 2 GPUs on PCIe

PIPELINE · PP

cheap wire,
empty stages



grey = stage waiting = the bubble
batch 1: ≈4× slower · batch 64: ≈no loss

One request = 4 stages in series: only 1 of 4 GPUs works per token.

Gain: comm = 1 P2P hop per stage — crosses PCIe/Ethernet fine
Cost: bubble ≈ 1/(stages+batch) — dies at low concurrency

EXPERT · EP

big GEMMs,
spiky traffic



one hot expert stalls every rank
DeepSeek serving: DP-attn + EP-FFN (TP=1)

Whole experts live on distinct GPUs; a token visits only k of N — bigger GEMMs, but tokens must travel to find experts.

Gain: best scaling for MoE at large batch — the DeepSeek recipe

Crossover: NVLink in-node → TP. Across hosts → TP × PP. MoE + fat fabric → DP-attention with Wide-EP (NVIDIA NVL72, per-node AMD).

Numbers from vLLM docs, SqueezeBits & AMD ROCm benchmarks — always re-measure with your own traffic.

LLM INFERENCE PARALLELISM · SLIDE 3

Beyond one box

Across hosts the question changes: how do whole request lifecycles get spread over a cluster?

One giant model: TP × PP

all-reduce stays on NVLink, only stage hops cross the wire



Fits one GPU: replicas + router

the boring default — linear throughput, zero sharding math



Prefill/decode disaggregation — the serving-cluster pattern

prefill is compute-bound, decode is bandwidth-bound — never let them share a GPU



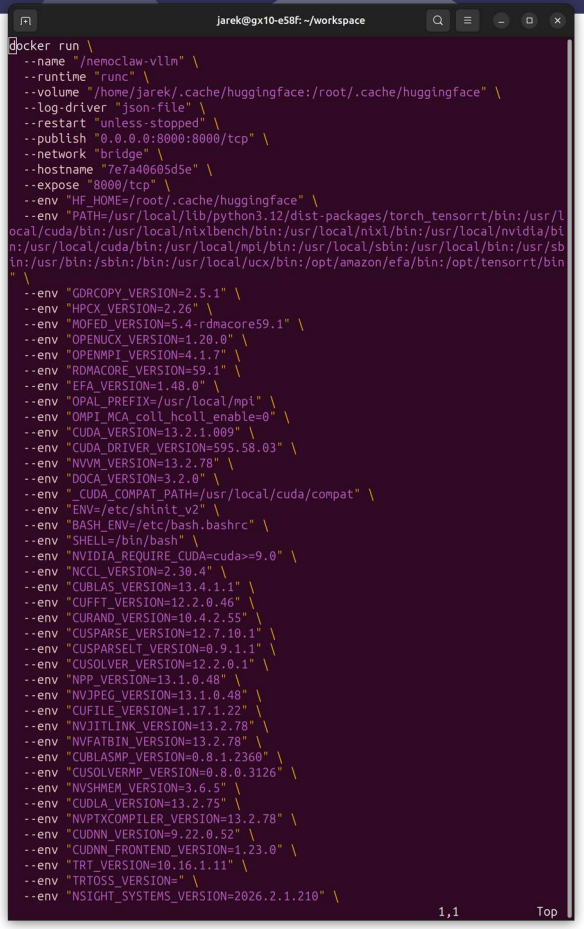
Who runs this: DeepSeek (3 prefill + 9 decode nodes), Meta, Mistral, Perplexity — via vLLM/SGLang + NVIDIA Dynamo (GTC 2025).

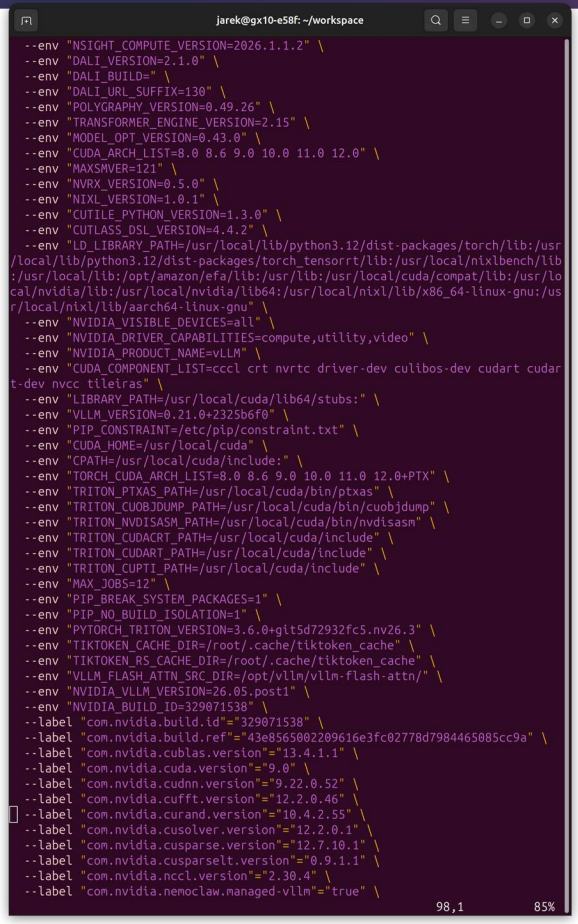
Same idea at rack scale: Wide-EP — one MoE instance spans an entire NVL72 NVLink domain.

Setting up inference

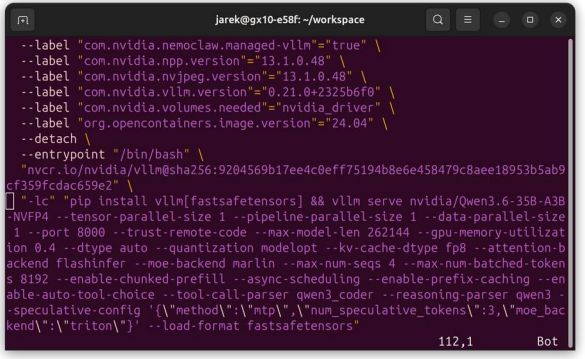
- Entry level (Ollama, LM Studio,…)
  - Hot model load and unload
  - Model catalogues
  - Simple config but it means lack of optimisation for a given platform
  - Generally not very safe (calling home, unencrypted logs with context...)
- Manual / professional grade (VLLM, llama.cpp, sd_lang, ...)
  - Hundreds of runtime flags, need knowledge/experimentation
  - Usually run with models available on HF
- Community recipies and engine forks (r/localllama)
  - Often available for popular models / hardware
  - Strata – Q3 2026 revolution

Manual setup





Example docker run command for VLLM. Source Nvidia - nemoclaw



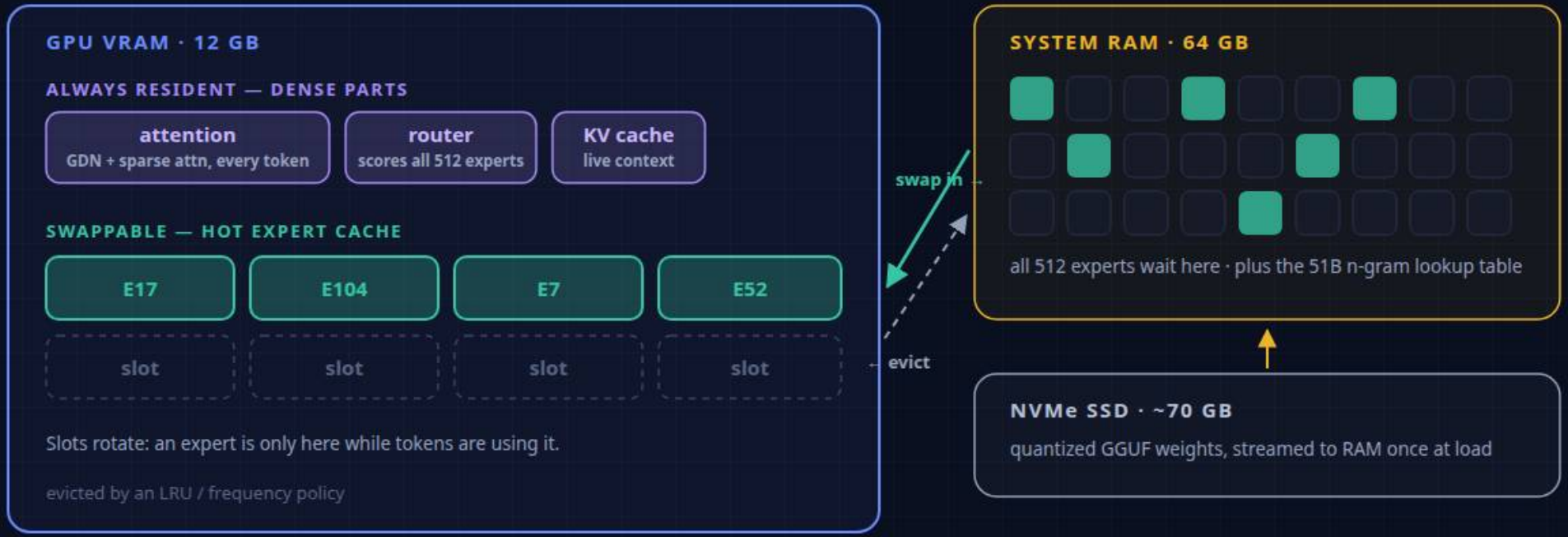
Community resources

- Sometimes apart from config are based on forks of vllm or llama.cpp
- Usually squeeze almost everything from your device
- So far from my experience 2x better performance than Ollama with similar quantization or uneducated manual config
- blazux, peonist-ai – thank you!!

INFERENCE ENGINE · STRATA

Strata: what lives in VRAM — and what flows

Dense pieces stay on the GPU permanently; experts are streamed in from RAM as the router needs them.



Why it works: MoE is sparse — each token uses only ~10 of 512 experts, so VRAM holds a rotating cache instead of the whole model.

Result: a 125B-parameter Qwen model on a 12 GB consumer GPU — dense parts fixed, experts in motion.

Hardware

- Dedicated GPUs
  - EGPU
  - Ex Data Centers / Off-lease cards
- Unified Memory
  - DGX/RTX Spark
  - AMD Strix/Medusa Halo
- What is cooking in China

Dedicated GPU

- Not gonna go through models, it would take half of the presentation time. You know how to read params now.
- Extrenal GPU
  - Requires USB/Thunderbolt 4
  - If entire model fits GPU it is as fast as motherboard slot
  - Also available for MAC
- Decommisioned GPUs
  - No video ports, need external cooling
  - Larger power consumption than in modern cards
  - Hard to say how much life left in them

Unified memory devices comparison

Device	Nvidia DGX / RTX Spark	AMD Strix / Medusa Halo	Mac
May run in parallel	**DGX -Yes up to 200 GB/s,** RTX – depends on I/O provided by OEM	Yes, speed depends on I/O provided by OEM	Yes, via ethernet port
eGPU	No USB-3	Depends the OEM (USB-4)	Apple agreed, but I don’t know the status
Good gaming rig	(not sure)	**Yes**	**Yes**
Dynamic CPU/GPU memory split	**Yes**	Generally yes, but some software may explicitly require VRAM not GTT	**Yes**

Strix Halo – AMD Ryzen™ AI Max+ 395/495

- Enormous community support
  - Halogram, GUFO inference engines
- First reports of GUFO forks utilising NPU + GPU
  - Prefill on par with DGX Spark!
- Some OEMs give 2xUSB4 v2(80GB/s) + 4xUSB4(40GB/s)

Is it worth it?

- Privacy
  - Still need internet access
  - Entry level inference engines not so private / safe
- Fine-tuned / uncensored models – double edge sword
- It’s not plug and play if you want too squeeze your hardware
- Learning experience / CV entry valuable for only for:
  - Cloud devops engineers
  - Inference engine developers
  - GPU driver / firmware devs

Is it worth it - costs

- Very conservative estimation method: I use bit less than $100 worth of tokens mothly (Qwen 3.8 Flash Next from Alibaba Cloud). So ~$1000 per year.
- More realistic usage is an equivalent of single $200 frontier lab subscription (no limits but slower tps). So $2400 per year.

Device	Price (vendors)
Cheapest 24GB card (Intel Arc Pro B60)	$1200
Cheapest 32GB card (Radeon AI PRO R9700)	$2200
Cheapest Strix Halo	$3000
DGX Spark	$6000
Nvidia RTX5090	$10000

Is it going to be worth it?

- ~~Hardware prices not expected to drop until 2029/30 :(~~
- Coreweave and Chinese burst – will it affect prices soon?
- How long will Chinese companies drop new near-frontier models?
- What will be the requirements of future models?
- Regulations?